ANTFLY · DOCUMENT ENRICHMENT · 1 / 4

From PDF bytes to searchable pages.

How Antfly turns a PDF into OCR text, chunks and page vectors — without ever buffering the whole document, sharing a mutable parser across threads, or letting one bad page sink the rest.

Inspect & pick OCR candidates
Born-digital pages that pass quality checks never render.

1

PREPARE
PreparedDocument

2

Parse once, fork per worker
Immutable RenderForkTemplate → private readers.

Bounded multi-page render call
Request-ordered results, one payload or failure each.

3

RENDER
zig/lib/pdf

4

Admission-controlled parallelism
min(requested, operator, memory, pages left).

Fill one render/inference window
Item, byte, pixel & working-set caps — never whole-doc.

5

SCHEDULE
enrichment_runtime

6

Binary media handoff
Borrowed RGBA8 / attachments — no base64 JSON.

Execute the native batch
LoadedReader.readBatch · batched encoder + KV decoder.

7

INFER + PUBLISH
inference Node

8

Merge by page_number & persist
Quality-gated selection, then units, chunks, vectors.

~51k
lines of pure Zig in `zig/lib/pdf` — parser, fonts, decoders, rasterizer

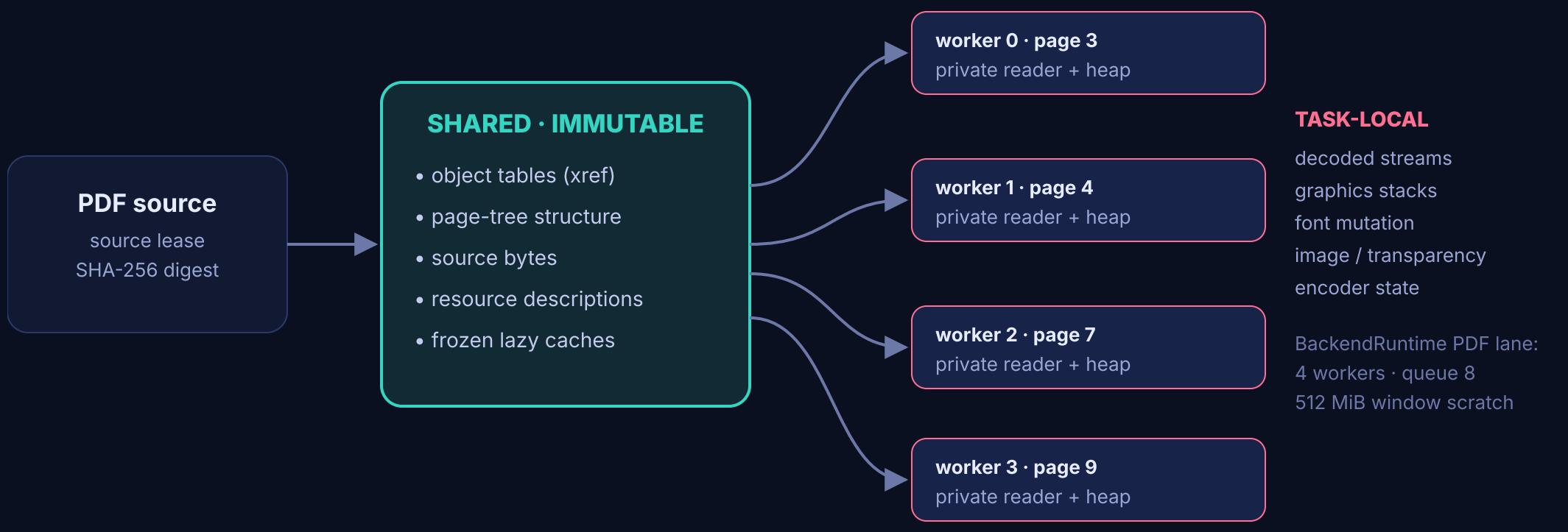
8
pages per default OCR window, matching Florence's native batch

2,048
default page ceiling per document (hard max 16,384)

1
render window retained per doc (+1 admitted prefetch)

One parse. Many private renderers.

`PdfRenderSession` isn't thread-safe — so it isn't shared. It discovers the page tree once, freezes an immutable `RenderForkTemplate`, and every worker calls `Reader.forkForRendering` to get its own allocator, caches and graphics state.



What the Zig reader decodes

Stream filters

`FlateDecode` `LZWDecode` `ASCII85` `ASCIIHex` `RunLength` `DCT (JPEG)` `JPX (JPEG 2000)` `JBIG2` `CCITTFax`

Fonts

`Type1 (cleartext + eexec)` `TrueType` `CFF` `OpenType/CFF` `Type3` `Type0 composite` `ToUnicode + ligatures`

Simple Type1/TrueType fonts stay on one-byte codes even when an oversized ToUnicode codespace suggests otherwise; Type0 keeps multibyte decoding. Image streaming keeps native RGBA plus predictor rows instead of full decoded colour & soft-mask planes — a 98.7 MB decoded image that used to fail the 64 MiB stream limit now renders pixel-exact.

Every page is priced before it renders

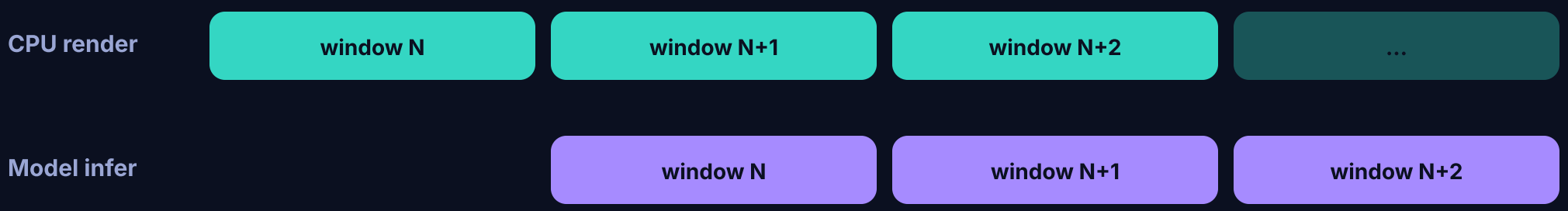
```
page_weight =
  RGBA output
 + resampling scratch
 + drawing / compositing scratch
 + bounded stream-decode reservation
 + encoded PNG estimate
 + retained request bytes
```

Estimates are conservative; a hard `BudgetedAllocator` enforces the truth. A tighter pixel cap **lowers DPI rather than rejecting** a page. Scratch pressure only reduces concurrency — it never changes prepared geometry. Each doc records its measured scratch peak + 12.5% as a hint for the next window.

`render_dpi / render_resolution`	Rasterize at requested DPI, then model preprocessing (`model_input` is opt-in)	**150 · requested_dpi**
`max_rendered_pixels / dimension`	Per-page spatial caps	**40 MP · 4096 px**
`OCR_RENDER_INFLIGHT_PIXELS / BYTES`	One concurrent render wave	**50 MP · 256 MiB**
`OCR_RENDER_PARALLEL_PAGES`	Never derived from CPU count; clamped 1..8	**1**

Render the next window while the model reads this one.

Rendering never races ahead of inference across the whole document. A double-buffered loop overlaps CPU rendering of window N+1 with GPU inference on window N — and the second window must win its own composite memory lease first.



prefetch_batches = 1 (default for OCR + visual embedding)
prefetch_batches = 0 → strict acquire · render · infer · release
>1 → clamped (would recreate whole-doc buffering)

Deadlock-free resource order

1. **Document extraction working-set lease**
   parse, plan, coordinator state — all budgeted
2. **Render-window lease**
   scratch may be partially granted; output peak required in full
3. **Inference queue units**
   only after the window is ready — never held while waiting on renderers

Released in reverse. If inference is saturated, rendering simply stops at the admitted window.

Two-level scheduling

Antfly owns the logical queue. Requests are partitioned by execution compatibility *before* media is materialized: interleaved `A,B,A,B` become `A,A` + `B,B`. Compatibility = model generation, task, provider scope, prompt/schema, input representation and resource class — not just model name.

The inference node owns physical execution. It fuses eligible work across multiple Antfly clients, admits against the loaded model and device, and dispatches through the task's executor. Borrowed page attachments carry item/source/page identity, so pages from *different documents* can share one native batch.

Inside a native Florence batch



fallback ladder: batched KV → full batched decoder → serial reader (documented errors only) · result count must equal image count

OCR has to earn its place on the page.

Results are joined by `page_number`, never completion order. A page can't be persisted as OCRed until its output passes validation *and* beats the embedded text on quality.

Selection rules

- Empty embedded text can't beat meaningful OCR.
- Empty, punctuation-only and **prompt-echo** OCR is always rejected.
- A short transcription can't replace substantial embedded content.
- Both pass quality? **Keep the embedded text** — "longer" isn't "better".
- Quality sampling uses bounded, deterministically offset intervals: a spaced heading can't condemn good body text; a clean prefix can't hide later corruption.
- Selected-but-weak OCR is flagged `ocr_selected_low_quality`.

Three failure classes

Per-page permanent

Malformed content stream, over-limit page, trivial OCR. Recorded; valid siblings continue.

Batch / provider

Count mismatch, unsupported shape. Only that microbatch retries serially — with the reason recorded.

Systemic retryable

Capacity, shutdown, memory pressure. Cancel *and join* every render task, then hand back to the durable enrichment worker.

"Embed this PDF" is never one implicit vector

Text chunks	Page images	Both	Doc-level
one vector per extracted chunk	one visual vector per rendered page	separate named spaces, never mixed	only via a named, versioned reducer

Page-image identity `antfly-pdf-page-embedding-v4` hashes the source digest & page, DPI & resolution policy, pixel/dimension caps, renderer ceilings, decoder limits, transport, and resolved image transform. Change any of them and staged + published vectors are invalidated. One missing vector fails the whole document attempt — partial responses can't silently shrink what's searchable.

462
PDF tests passing in Debug and ReleaseFast

51/51
pages, forced-OCR qualification cohort (academic, admin, law) — zero OCR failures

0 px
diff vs. serial reference at 150 DPI (1241×1754) with streaming images

12
required invariants — e.g. a batch must never silently become serial

Qualification: scripts/bench/pdf/RESULTS-2026-09-09-STREAMING.md (internal engineering checks, not accuracy claims)